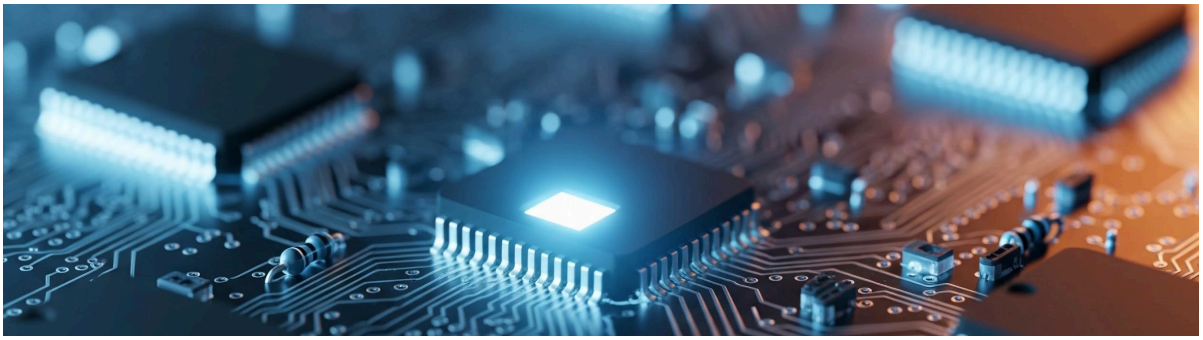


Trabajo Práctico 1

Procesamiento de Imágenes I

Integrantes

- Amanda Tardugno
- Lucas Díaz Celauro
- Adriano Marzol Perini

Carrera

Tecnicatura Universitaria en Inteligencia Artificial
Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Universidad Nacional de Rosario

20/04/2026

Ejercicio 1: Ecualización local de histograma

Descripción del Ejercicio

El objetivo de este ejercicio es implementar un algoritmo de Ecualización Local de Histograma para mejorar el contraste de detalles ocultos en imágenes con iluminación no uniforme. A diferencia de la ecualización global, que busca un histograma uniforme para toda la imagen, la ecualización local ajusta cada píxel basándose en la distribución de intensidades de su vecindad inmediata (ventana), permitiendo realzar texturas y formas que de otro modo quedarían saturadas o subexpuestas.

Análisis de Problemas y Evolución de la Técnica

Durante el desarrollo de la solución, la implementación atravesó una refactorización crítica para solucionar artefactos visuales:

1. **Artefactos de Bloque (Efecto Mosaico):** La implementación inicial procesaba la imagen dividiéndola en bloques estáticos. Se calculaba y aplicaba el histograma a todo el bloque a la vez. Esto generaba discontinuidades artificiales muy marcadas en los bordes de cada recuadro, ya que no existía una transición estadística suave entre un bloque y sus vecinos.
2. **Migración a Ventana Deslizante:** Para solucionar esto, se modificó el algoritmo para avanzar píxel a píxel (paso unitario). La estadística se recalcula para la vecindad de cada píxel, y la ecualización local resultante se aplica *únicamente* al píxel central. Esto garantiza una transición continua del contraste.
3. **Tratamiento de Bordes:** El uso de una ventana deslizante requirió implementar un relleno (*padding*) para poder procesar los píxeles periféricos de la imagen. Se utilizó la técnica de **Reflexión** (cv2.BORDER_REFLECT) en lugar de un relleno con ceros (negro). Rellenar con negro habría alterado el histograma local en los extremos, oscureciendo artificialmente los bordes de la imagen resultante.
4. **Suavizado de la imagen:** Al trabajar con un ecualizado local, el más mínimo ruido produce un gran efecto. Por ello, al finalizar el ecualizado decidimos aplicar un suavizado (cv2.medianBlur) para suavizar el efecto del ruido. En la figura 1 se puede observar la imagen original, con el ecualizado, y con el ecualizado + suavizado.

Figura 1: Comparación de enfoques. Izquierda: Imagen original. Centro: Imagen ecualizada localmente con ruido aumentado. Derecha: Implementación final utilizando suavizado para eliminar el ruido luego del ecualizado local.

Validación de parámetros

A nivel de código, se añadió una validación estricta para garantizar que las dimensiones de la ventana proporcionadas como parámetro sean **impares** (ej. 5x5, 15x15).

Si se permitiera una ventana de dimensiones pares (ej. 4x4), la misma no poseería un centro geométrico unívoco. La asignación del valor ecualizado sufriría un desplazamiento espacial (*shift*) de medio píxel, desalineando sutilmente la imagen procesada respecto a la geometría original de la escena.

Resultado de la Ecualización de Histograma Local

Al aplicar el algoritmo a la imagen con detalles escondidos, como muestra la figura 1, logra resaltar los detalles en lo que pareciera ser 5 cuadrados negros antes de ecualizar: en el superior izquierda se encuentra un cuadrado, en el superior derecho una barra diagonal, en el centro una letra “**a**”, en el inferior izquierdo 4 barras horizontales, y en el inferior derecho un círculo.

Influencia del Tamaño y Forma de la Ventana

El algoritmo depende críticamente de las dimensiones de la ventana de vecindad ($W \times H$). Para analizar esta dependencia, se realizaron pruebas con ventanas cuadradas de distintos tamaños, una ventana asimétrica y una ecualización global como línea base:

- **Ecualización Global:** Mejoraría el contraste general de la imagen, aunque no lo hace para esta imagen en particular. Falla en realzar los detalles más sutiles en áreas que comparten un mismo rango tonal a nivel macro.
- **Ventana Pequeña (5x5):** Muestra una extrema sensibilidad a los cambios locales, pero al poseer una muestra estadística muy reducida (25 píxeles), amplifica severamente el ruido de alta frecuencia (efecto de "sal y pimienta").
- **Ventana Mediana a Grande (21x21 y 51x51):** Representan puntos de equilibrio. Al aumentar la ventana, el contraste se realza de forma natural y orgánica, mitigando la

ganancia de ruido al promediarlo en una superficie mayor. Cuanto mayor la ventana, más contraste se pierde en los bordes.

- **Ventanas Asimétricas (5x51 y 51x5):** Al utilizar una ventana estrecha en una dirección y alargada en la otra, el comportamiento del filtro se vuelve anisotrópico. Como se observa en los resultados, esta configuración promedia y difumina los detalles en el eje alargado, generando un efecto de barrido direccional, mientras que mantiene un alto contraste reactivo en el eje horizontal, aunque genera ruido en esa dimensión en caso de ser una ventana pequeña. Es notable este efecto en los detalles de la imagen escondida en el cuadrado inferior izquierdo.

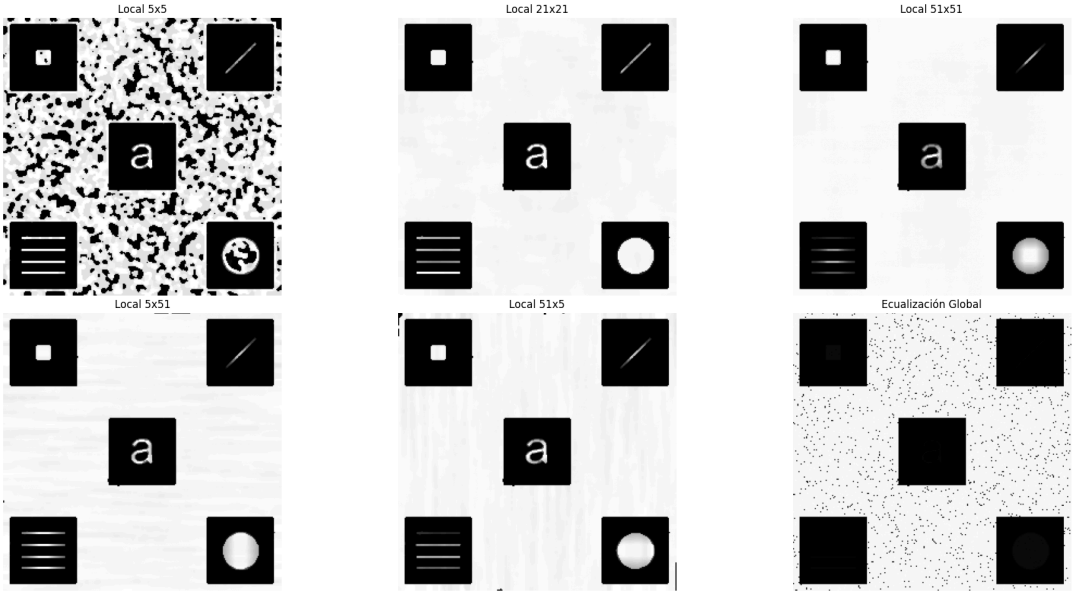


Figura 2: Análisis de sensibilidad de la ecualización. Se observa la progresión desde el ruido en ventanas pequeñas (5x5) hasta el equilibrio en ventanas mayores (21x21, 51x51). También se expone el efecto direccional de una ventana asimétrica (5x51) contrastado contra una ecualización de histograma global convencional.

Conclusiones

La Ecualización de Histograma Local (LHE) mediante ventana deslizante ha demostrado ser significativamente superior a la ecualización global y al procesamiento por bloques para la recuperación de información.

El éxito del algoritmo requiere considerar un *trade-off*: ventanas pequeñas maximizan el contraste de microtexturas a costa de amplificar el ruido, mientras que ventanas grandes ofrecen un contraste más limpio pero diluyen los detalles cercanos a bordes fuertes. Finalmente, la correcta gestión espacial, forzando dimensiones impares y reflejando los bordes, es indispensable para evitar artefactos debido al algoritmo.

Ejercicio 2: Corrección automática de múltiple choice

Paso 1: Segmentación de celdas (`bordesExamen`)

Técnica

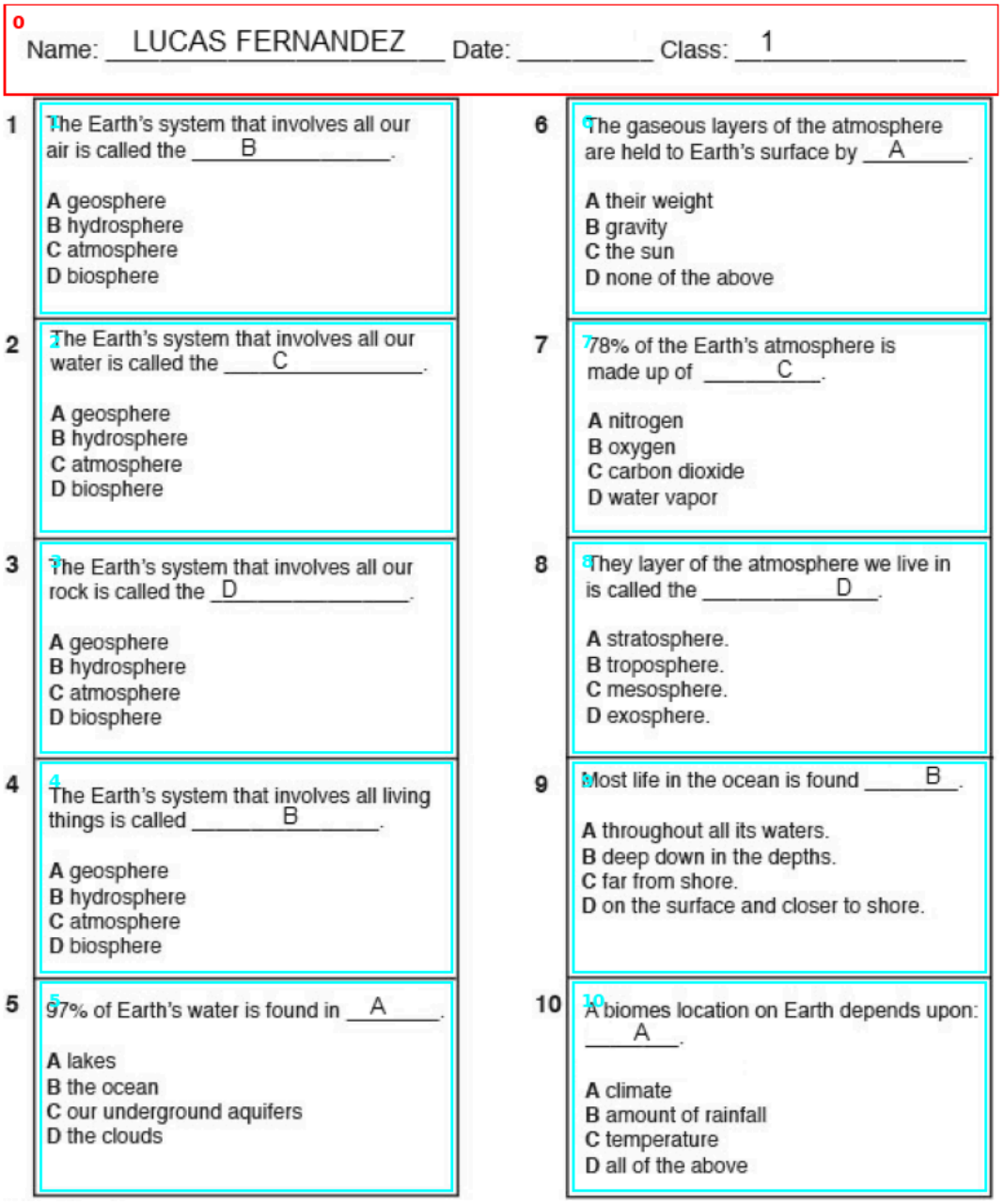
Se binariza la imagen (`img < 192`) y se suman los píxeles negros por columna y por fila (proyección vertical y horizontal). Como las líneas negras de la grilla tienen muchos más píxeles que cualquier otra fila o columna, aplicando un umbral alto (75% del alto o ancho de la imagen) quedan solo las coordenadas de esas líneas.

Luego, recorriendo las transiciones entre "línea" y "no línea", se identifican los bordes izquierdo, derecho, superior e inferior de cada celda.

Desafíos

- **Grilla asimétrica:** el layout de las 10 preguntas es de 2 columnas × 5 filas, por lo que hubo que filtrar manualmente las coordenadas correspondientes a los bordes externos de la tabla (las primeras y últimas líneas detectadas) para quedarnos solo con las líneas internas.
- **Líneas con más de 1 px de ancho:** en lugar de buscar picos exactos, se detectan los *flancos* (transición 0→1 y 1→0) sobre el vector umbralizado. Eso permite identificar el comienzo y el fin de cada línea aunque tenga grosor variable.
- **Padding de seguridad:** se agrega un margen de 2 px hacia adentro de cada celda para que no queden píxeles de la línea divisoria dentro del recorte.

1. bordesExamen: rojo = encabezado (0), cyan = 10 preguntas (1..10)



0 Name: LUCAS FERNANDEZ Date: _____ Class: 1

1 1 The Earth's system that involves all our air is called the B.

A geosphere
B hydrosphere
C atmosphere
D biosphere

2 2 The Earth's system that involves all our water is called the C.

A geosphere
B hydrosphere
C atmosphere
D biosphere

3 3 The Earth's system that involves all our rock is called the D.

A geosphere
B hydrosphere
C atmosphere
D biosphere

4 4 The Earth's system that involves all living things is called B.

A geosphere
B hydrosphere
C atmosphere
D biosphere

5 5 97% of Earth's water is found in A.

A lakes
B the ocean
C our underground aquifers
D the clouds

6 6 The gaseous layers of the atmosphere are held to Earth's surface by A.

A their weight
B gravity
C the sun
D none of the above

7 7 78% of the Earth's atmosphere is made up of C.

A nitrogen
B oxygen
C carbon dioxide
D water vapor

8 8 They layer of the atmosphere we live in is called the D.

A stratosphere.
B troposphere.
C mesosphere.
D exosphere.

9 9 Most life in the ocean is found B.

A throughout all its waters.
B deep down in the depths.
C far from shore.
D on the surface and closer to shore.

10 10 A biomes location on Earth depends upon: A.

A climate
B amount of rainfall
C temperature
D all of the above

Paso 2: Validación del encabezado (`detectarCamposEncabezado`, `validarEncabezado`, `contarPalabras`)

Técnica

Sobre el recorte del encabezado se calculan las **componentes conectadas**. A cada componente se la clasifica como:

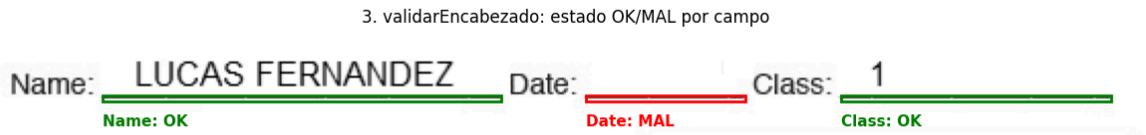
- **línea** si `ancho > 3·alto` y `ancho > 30` (las líneas subrayadas de los campos),
- **caracter** en cualquier otro caso.

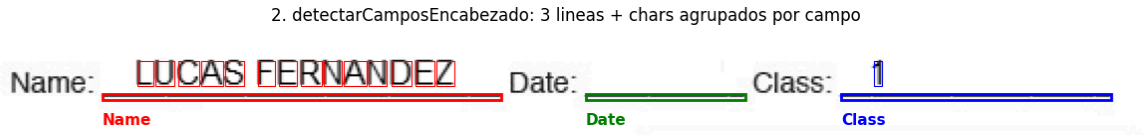
Se quedan las 3 líneas más anchas (Name, Date, Class) y se las ordena de izquierda a derecha. Luego, para cada línea se recolectan los caracteres que cumplan:

- el centro horizontal del caracter cae dentro del rango `[xl, xl+wl]` de la línea,
- el caracter está **arriba** de la línea (`y + h ≤ yl`),
- pero no demasiado lejos (`yl - (y+h) ≤ 25 px`).

Esas tres listas alimentan la validación:

- **Name:** 1 ≤ chars ≤ 25 y **≥ 2 palabras**.
- **Date:** exactamente 8 caracteres formando **1 palabra**.
- **Class:** exactamente 1 caracter.





Cómo contar palabras:

Esta fue la parte más espinosa del ejercicio. La idea base es simple: ordenar los caracteres por `x`, medir el *gap* entre el final de un caracter y el inicio del siguiente, y decidir que un gap grande equivale a una frontera de palabra. El problema es elegir el umbral.

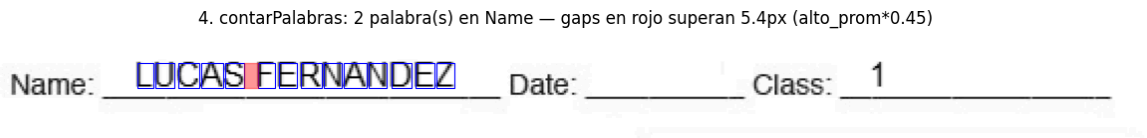
Intento 1: umbral = ancho promedio del caracter.
Fallaba en casos como `MARIA LOPEZ`: el ancho promedio era 8.9 px, pero el gap real entre palabras era solo 6 px. Al ser `6 < 8.9`, todas las palabras del Name se leían como una sola, así que Name siempre daba MAL.

Intento 2: umbral = ancho promedio × 0.5.
Se arreglaron los Names, pero se rompieron los Dates. En una fecha como `11/07/24`, los caracteres `1` y `/` son muy angostos (4–5 px). El ancho promedio bajaba a 5.9, dando un umbral de 2.95. El gap entre los dos `1` iniciales era de 5 px (el `1` ocupa solo el centro de su bounding box), así que `5 > 2.95` y el algoritmo inventaba una frontera de palabra inexistente, haciendo que Date diera MAL.

Intento 3 (final): umbral = alto promedio × 0.45.
El **alto** de los caracteres es mucho más uniforme que el ancho (todos los caracteres miden aproximadamente la misma altura, sin importar si son `1`, `M` o `/`). Además, en tipografía un espacio entre palabras suele ser aproximadamente la mitad del alto del caracter, lo que lo vuelve una referencia natural. Con alto promedio ≈ 12 px, el umbral queda en ≈ 5.4 px.

Caso	Gap máximo intra-palabra	Gap entre palabras	Resultado
`11/07/24`	5		1 palabra
`MARIA LOPEZ`	2	6	2 palabras
`ESTEBANALVAREZ`	2		1 palabra

Cuando una métrica varía mucho según el contenido (ancho del caracter), su promedio no es buen umbral. Conviene anclarse en algo estable de la tipografía; en este caso, la altura.



Otros desafíos

- **Identificar qué componente es "la línea" del campo:** se resolvió por relación de aspecto (`w > 3·h` + ancho mínimo). Esto mismo excluye al `:` de los labels y a los caracteres normales.
- **Asignar cada caracter a su campo:** se usa el centro horizontal del caracter (no el borde izquierdo), porque algunos caracteres se extienden ligeramente fuera del rango de la línea por rasgos tipográficos.
- **Distinguir labels (`Name:`, `Date:`, `Class:`) de los valores:** los labels están a la **izquierda** de cada línea, no arriba, así que el filtro por rango horizontal los descarta automáticamente.

Paso 3: Recorte del campo Name (`obtenerName`)

Reutiliza `detectarCamposEncabezado` para ubicar la primera línea (Name) y los caracteres que están sobre ella. Luego calcula el bounding box mínimo que contiene a todos esos caracteres y lo expande con un margen proporcional al *gap* entre los caracteres y la línea, para que el recorte se vea balanceado y no pegado al texto.

Paso 4: Detección de la letra de respuesta (`encontrarRespuestas`)

Intento descartado

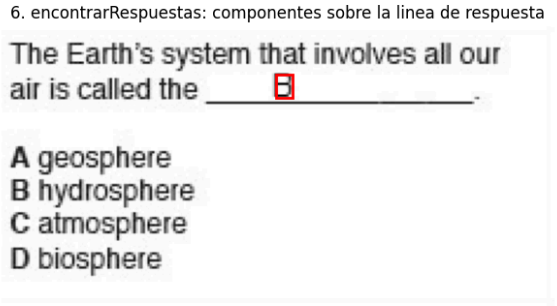
Inicialmente se intentó ubicar la línea de respuesta contando los espacios en blanco entre los renglones de las opciones (sabiendo que siempre hay 4 opciones y la línea de respuesta). Se descartó rápidamente: el ancho del texto de las preguntas variaba y en algunos casos se mezclaba con la línea punteada de la opción, perdiendo robustez.

Solución final

Sobre el recorte de cada pregunta se aplica `connectedComponentsWithStats` y se busca un componente que sea **mucho más ancho que alto** (`w > 3·h`). Ese es la línea de respuesta y sirve como *ancla*. Luego se buscan los componentes que:

- estén **dentro** del rango horizontal de la línea,
- y cuyo bounding box **intersecte verticalmente** la línea (la letra escrita apoya su trazo sobre la línea).

Se filtran por área mínima para descartar puntos o manchas.



Paso 5: Clasificación A / B / C / D (`extraerCaracteristicasLetra`, `identificarRespuestas`)

Intento descartado

Primero se probó clasificar por el ancho, alto o área del bounding box. Los scatter plots de calibración (ver `calibrar_letras.py`) mostraron que las cuatro letras tienen dimensiones muy similares y se superponen, por lo que cualquier variación mínima de caligrafía las confunde.

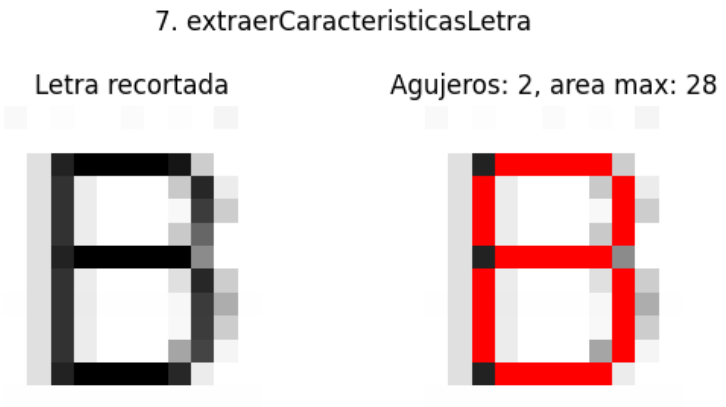
Solución final: criterio híbrido topológico + geométrico

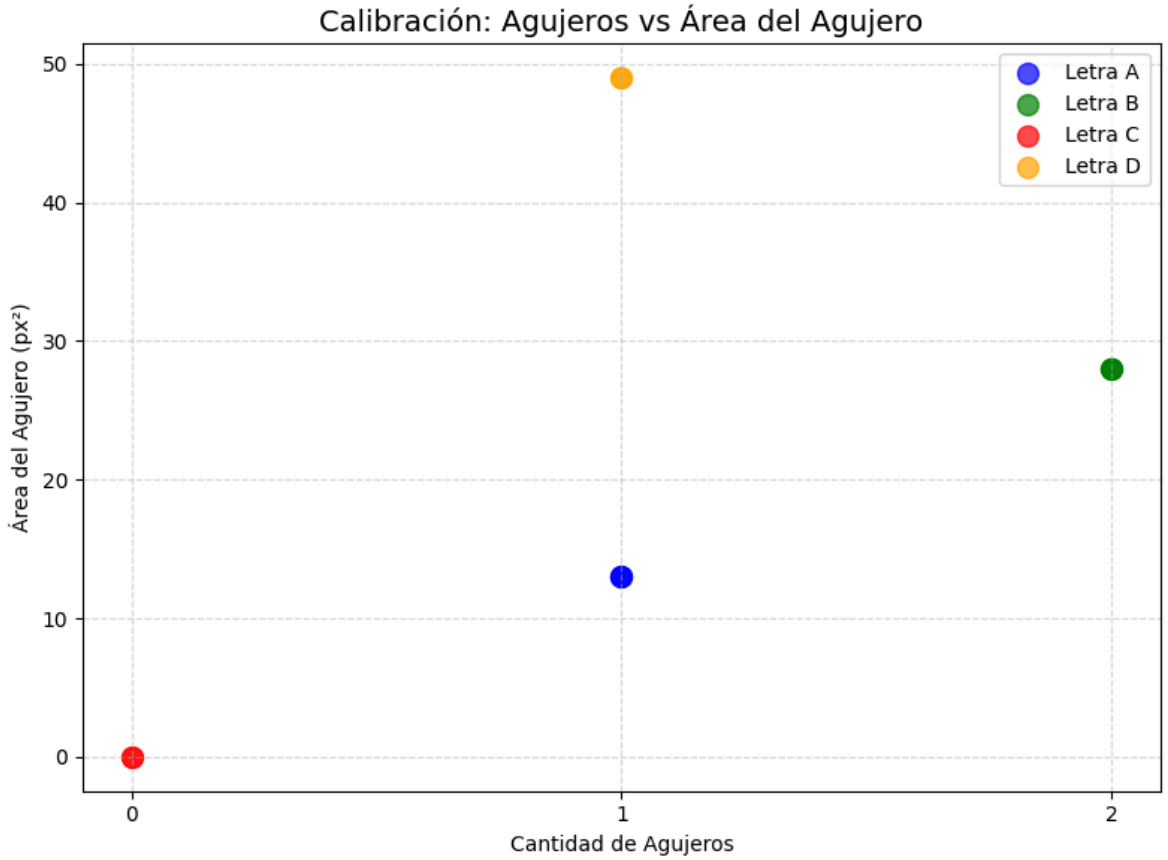
Con `cv2.findContours(..., RETR_TREE)` se obtiene la jerarquía de contornos. Un contorno con padre (`hierarchy[i,3] != -1`) es un **agujero** dentro de la letra. Las cuatro letras se distinguen así:

Letra	Cantidad de agujeros	Criterio adicional
C	0	—

B	2	—
A	1	agujero pequeño (≤ 30 px^2)
D	1	agujero grande (> 30 px^2)

El umbral de 30 px² se calibró visualmente con el scatter plot de `calibrar_letras.py` sobre `examen_4.png`, que tiene las 4 letras representadas.





Limitaciones asumidas

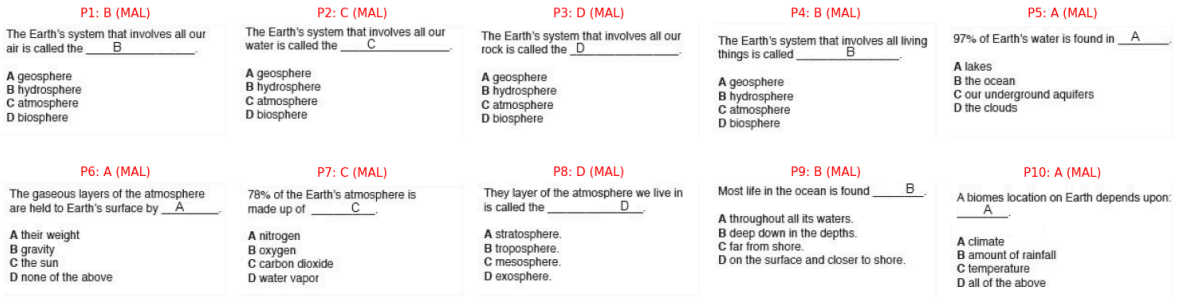
- Si el alumno escribe una letra que no es A/B/C/D, la clasificación puede dar un resultado espurio: una `E` o `F` tiene 0 agujeros y se clasificaría como C; una `O` tiene 1 agujero grande y se clasificaría como D.
- Si la letra está poco cerrada (caligrafía abierta), el agujero podría no detectarse y la clasificación falla.

Paso 6: Evaluación final y reporte (`tp1Ej2.py`)

Corrección

Por cada examen se arma un bucle sobre las 10 preguntas: se llama a `identificarRespuestas` y se compara con la lista `respuestasCorrectas`. Si la respuesta detectada es `None` (no se pudo clasificar, se marcaron múltiples opciones o no se marcó ninguna), la pregunta cuenta como `MAL`. Se imprime el estado pregunta por pregunta y el total de aciertos.

8. identificarRespuestas + comparacion: 0/10 aciertos



Procesamiento por lotes

Se recorre `examen_1.png` a `examen_5.png` con un simple `for n in range(1, 6)`. Se priorizó la legibilidad sobre usar `glob` o similares.

Canvas final

Se construye una imagen BGR de 500×1000 px (5 filas × 100 px de alto) y por cada examen se:

- dibuja un rectángulo verde (≥ 6 aciertos) o rojo (< 6),
- pega el recorte del *Name* redimensionado a 50 px de alto,
- escribe el texto: “tiene N aciertos. Está APROBADO/DESAPROBADO”.

Se eligió BGR (canales de color) para poder superponer los rectángulos de color sobre recortes que originalmente son en escala de grises.